M.TECH, 3RD SEMESTER PROJECT

A Safety-Gated LLM Auto-Remediation Agent for AIOps

Pre-execution verification of untrusted remediation proposals, with a measured safety–autonomy trade-off

`sentinel-remediate`

System Architecture & Evaluation Report

Submitted by

[Student name] · [Roll number]

Under the guidance of

[Project guide]

[Department]

[Institute]

08 October 2026 · generated from repository commit `9bfac6c`

Abstract

Large language models can now read an incident's logs, metrics, traces and topology and propose a remediation, but they cannot be trusted to execute it: a fluent, plausible proposal may cordon a healthy node, restart a shared database or switch on a fault-injection flag. This project builds and evaluates **sentinel-remediate**, a remediation agent in which the LLM is treated strictly as an *untrusted planner*. Every proposed action passes through a layered **pre-execution verifier** before it can reach the environment: a deterministic action envelope (allow-list, typed parameters, blast-radius and namespace policy), an independent LLM judge, and a learned, calibrated risk model, combined by a single risk threshold τ that trades safety against autonomy.

The system is built to run end-to-end with no Kubernetes cluster and no API key. It uses an offline environment of 41 incidents — 6 hand-built and 35 converted from IBM's public ITBench-Lite SRE scenarios — with a provenance trail on every ground-truth label. A seeded evaluation harness writes every run to DuckDB and regenerates every table and figure from scratch.

Over 7,585 runs, with $\tau = 0.5$ and a simulated planner, the share of runs in which a **destructive action reached execution** falls from 33.2% (ungated agent) and 21.5% (envelope only) to **1.0%** with the learned layer, which stops 95.5% of the destructive proposals that pass the envelope. Escalating every block costs autonomy, but retrying blocked actions with the verifier's reasons restores resolution to 92.7% at 92.2% net-safe resolution, against 66.8% for the ungated agent. The report sets these results against a type-only baseline and states the threats to their validity: simulated proposals, rule-derived labels, and curated topology.

Keywords: AIOps, incident remediation, LLM agents, guardrails, pre-execution verification, calibrated risk, safety–autonomy trade-off, ITBench.

Contents

1 Introduction 1

1.1 Problem 1

1.2 Approach 1

1.3 Contributions 1

1.4 Scope 2

2 Background 3

2.1 LLM agents for incident remediation 3

2.2 Guarding agent actions 3

2.3 The safety–autonomy trade-off 3

3 Requirements and Design Principles 4

4 System Architecture 5

4.1 Overview 5

4.2 Component inventory 6

5 Data Contracts 7

6 Component Design 8

6.1 Environment adapter 8

6.2 Context builder and telemetry tools 8

6.3 Diagnosis agents 8

6.4 The verification layers 9

6.5 Executor 11

6.6 Orchestrator loop 12

6.7 Run store 12

6.8 Gateway and console API 13

6.9 Observability and deployment 13

6.10 Triage console 13

7 Dataset and Ground Truth 15

7.1 Incidents 15

7.2 ITBench conversion pipeline 16

7.3 Label taxonomy and provenance 16

7.4 Hard cases: why the dataset needed them 17

8 Evaluation Methodology 18

8.1 Metrics 18

8.2 Experimental grid 18

8.3 Uncertainty 18

8.4 Learned-verifier protocol 19

9 Results 20

9.1 Baselines 20

9.2 Which layer caught what 21

9.3 Escalation versus retry 21

9.4 The risk–autonomy Pareto 22

9.5 Learned verifier: discrimination and calibration 22

9.6 Robustness to planner quality and topology 23

10 Discussion and Threats to Validity 25

10.1 Threats to validity 25

10.2 Interpretation 25

10.3 Experiments stated as not run 25

11 Engineering Quality and Reproducibility 27

11.1 Reproducibility 27

11.2 Defects found by verification 27

11.3 Verification of the live demonstration 28

12 Limitations and Future Work 29

13 Conclusion 30

A Appendix: Configuration Reference 31

A.2 Repository layout 32

List of figures

Figure 1. System architecture 5

Figure 2. The verification pipeline implemented by CombinedPolicy 9

Figure 3. One run of the orchestrator loop 12

Figure 4. Containerised demonstration stack 13

Figure 5. The console in live mode on itbench-sre-scenario-12 14

Figure 6. From raw ITBench snapshots to reviewed fixtures 16

Figure 7. Incident-level cross-fitting 19

Figure 8. Prevention partition at $\tau = 0.5$ 21

Figure 9. Left: bad-action-prevented against false-block across τ 22

Figure 10. Calibration of the learned verifier's out-of-fold P(destructive) 23

Figure 11. Agent-quality ablation 23

Figure 12. The console's results screen in live mode, rebuilt from the gateway's DuckDB rows by the same report code that produced this document 28

List of tables

Table 1. Build phases and where each is described 2

Table 2. Design principles and where they are enforced 4

Table 3. Components, their responsibilities and their locations in the repository 6

Table 4. Core contracts (src/sentinel/schemas) 7

Table 5. The action envelope (config/action_envelope.yaml) 10

Table 6. Learned-verifier features (verifier/features.py) 11

Table 7. Verification variants (verifier/policy.py: build_policy) 11

Table 8. Incidents by fault family (41 incidents, 17 families) 15

Table 9. Labeled candidate actions by provenance and label 17

Table 10. Hard-case rules 17

Table 11. Metric definitions 18

Table 12. Baselines at τ = 0.5 (escalate on block), every incident × every seed 20

Table 13. Prevention partition: shares of destructive proposals by the layer that stopped them 21

Table 14. What a block means: escalate to a human (blueprint default) versus retry with the block reasons as feedback (τ = 0.5) 21

Table 15. Out-of-fold classification of destructive actions (incident-level 5-fold cross-fitting) 22

Table 16. Agent-quality ablation: simulated planners proposing progressively more destructive actions 24

Table 17. Learned verifier with and without topology features (τ = 0.5, end to end) 24

Table 18. Threats to validity and how each is mitigated or measured 25

Table 19. Commands that reproduce each phase 27

Table 20. Defects found and fixed 27

Table 21. Key settings in config/default.yaml (the offline-evaluation overlay sets execution.auto_execute: true, dry_run: false) 31

1 Introduction

1.1 Problem

Site-reliability engineering is moving from runbooks toward agents: an LLM reads an incident's telemetry, forms a diagnosis, and proposes a remediation such as rolling back a deployment, scaling a service or disabling a feature flag. The diagnosis step is now often good. The hard part is the step after it. A proposal that is fluent and plausible can still be operationally wrong in ways that make the incident worse, for example:

- cordoning a healthy node,
- scaling a shared dependency to zero,
- restarting the database every service depends on, or
- switching on a fault-injection flag.

Letting such an agent act directly on a production control plane is unsafe. Making it ask a human every time throws away the autonomy that motivated it.

1.2 Approach

This project treats the LLM strictly as an **untrusted planner**. Proposal generation is separated from execution, and every proposed action must pass a **pre-execution verifier** before it can reach the environment. The verifier is layered:

- a **deterministic envelope** that never consults a model: allow-listed action types, typed and bounded parameters, a blast-radius cap, and forbidden namespaces;
- an independent **LLM judge** (v1);
- a **learned verifier** (v2) that outputs a calibrated probability that the action is destructive.

A single risk threshold τ combines the layers: an action executes only if the envelope passes it *and* its risk is below τ. Sweeping τ produces a measured **safety–autonomy trade-off** (a Pareto curve) rather than a single operating point.

1.3 Contributions

1. **A layered pre-execution verification architecture** for AIOps remediation. It is independent of the agent, records every layer's decision on every verdict, and enforces the safety rules in code: nothing executes without an `allow` verdict and an explicit configuration opt-in.
2. **An offline, cluster-free evaluation environment** of 41 incidents. Thirty-five of them are converted from IBM's ITBench-Lite SRE scenarios by a converter that records the provenance of every label and keeps ground truth out of agent-visible telemetry.
3. **A learned, calibrated risk model** evaluated with incident-level cross-fitting, and compared against a type-only baseline so that its gains are not overstated.
4. **A reproducible evaluation harness.** 7,585 seeded runs are stored as rows in DuckDB, and one command regenerates every number and figure in this report.

5. **A live triage console** that streams the loop step by step and shows the layered verdict. A headless-browser rehearsal of the demonstration checks it end to end.

1.4 Scope

The project studies *operational correctness*: whether a well-intentioned but fallible agent's action would harm the system. It does not study adversarial safety such as prompt injection or jailbreaks. All results in this report come from the offline environment with a simulated planner. The LLM-backed baselines are implemented and tested, but their evaluation is left as a cost decision (§10.3).

Table 1. Build phases and where each is described.

Phase	Deliverable	Section
P0	Pydantic contracts, offline environment adapter, deterministic envelope, fixtures	§5, §6.1, §6.4
P1	Diagnosis agents (ReAct, simulated), context builder, dry-run executor, LLM client + cost ledger	§6.2, §6.3, §6.5
P2	LLM judge, combined policy, orchestrator loop, DuckDB run store	§6.4, §6.6, §6.7
P3	Labeling pipeline, features, learned verifier, review CLI	§6.4, §7
P4	Evaluation harness: baselines, τ sweep, ablations, report figures	§8, §9
P5	Gateway API with live streaming, observability, triage console, containerised demo	§6.8–§6.10

2 Background

2.1 LLM agents for incident remediation

An AIOps remediation agent sees an incident through its observability signals:

- logs, metrics and traces;
- the service topology;
- firing alerts.

It must output an action against the control plane. Benchmarks such as Microsoft's AIOpsLab and IBM's ITBench [1] run these tasks against faults injected into microservice systems. Most of the agent's mistakes are not malicious. They are *wrong actions taken confidently*: the agent targets the component that shows symptoms instead of the cause, or picks an action whose blast radius is out of proportion to the fault.

2.2 Guarding agent actions

Three families of controls are common:

- **allow-lists and policy engines**: deterministic, auditable and cheap, but blind to whether a structurally valid action is right *for this incident*;
- **LLM-as-judge review**: semantically aware, but costly, non-deterministic, and itself fallible;
- **learned risk models**: cheap at inference and calibratable, but only as good as their labels.

This project composes all three and measures what each layer adds. The headline question is *which layer caught which destructive proposal*, reported as the prevention partition (§9.2).

2.3 The safety–autonomy trade-off

Any verifier trades two errors against each other:

- **missing** a destructive action (unsafe execution);
- **blocking** a correct one (false block, lost autonomy).

The project reports both sides at every threshold: bad-action-prevented against false-block, and resolution against τ. It also separates the verifier's own errors from the orchestration policy's response to a block (§9.3). In the results these turn out to matter as much as each other.

3 Requirements and Design Principles

Five rules govern the whole design. They are enforced in code and covered by tests, not just written down as conventions.

Table 2. Design principles and where they are enforced.

Principle	Enforcement
The LLM is an untrusted planner. No action reaches real execution without passing the verifier policy.	`Executor.run(dry_run=False)` raises `ExecutionRefused` unless it is given an `allow` verdict; rollback only *plans* an inverse action, which must itself be verified.
Real execution is opt-in. `auto_execute` is false everywhere by default.	Default config ships `auto_execute: false` (tested). The gateway returns HTTP 403 for `dry_run=false` unless the loaded config opts in.
Typed contracts. Every cross-module payload is a Pydantic model.	`src/sentinel/schemas`; the console's TypeScript types are *generated* from these models, never hand-written.
Offline first. The system runs end to end with no cluster and no API key.	`OfflineEnvAdapter`, the simulated planner, and lazy imports of the Anthropic SDK and cluster clients. A fresh clone passes the full test suite with no network.
Independence. The agent and the verifier share no state or prompts.	Separate prompts, separate LLM client handles and separate usage ledgers. The judge sees the proposal only as data. Ground-truth labels are attached *after* the verdict.

Two further requirements shape the evaluation:

- **reproducibility**: all randomness is seeded from config, every run records a `config_hash`, and `make eval` regenerates every figure from scratch;
- **persistence**: a result does not exist unless it is a run record in DuckDB.

4 System Architecture

4.1 Overview

Fig. 1 shows the components and how they connect. The **orchestrator** owns the loop. Inside it, an incident flows from the environment adapter through the context builder to the diagnosis agent. The agent's proposal then goes to the verifier policy, and only an `allow` verdict reaches the executor, which is the single path back into the environment. Blocks and revise decisions go back to the agent as feedback. Outside the loop, the run store persists every run; the evaluation harness and the gateway read from it; and the console and the observability stack sit in front of the gateway.

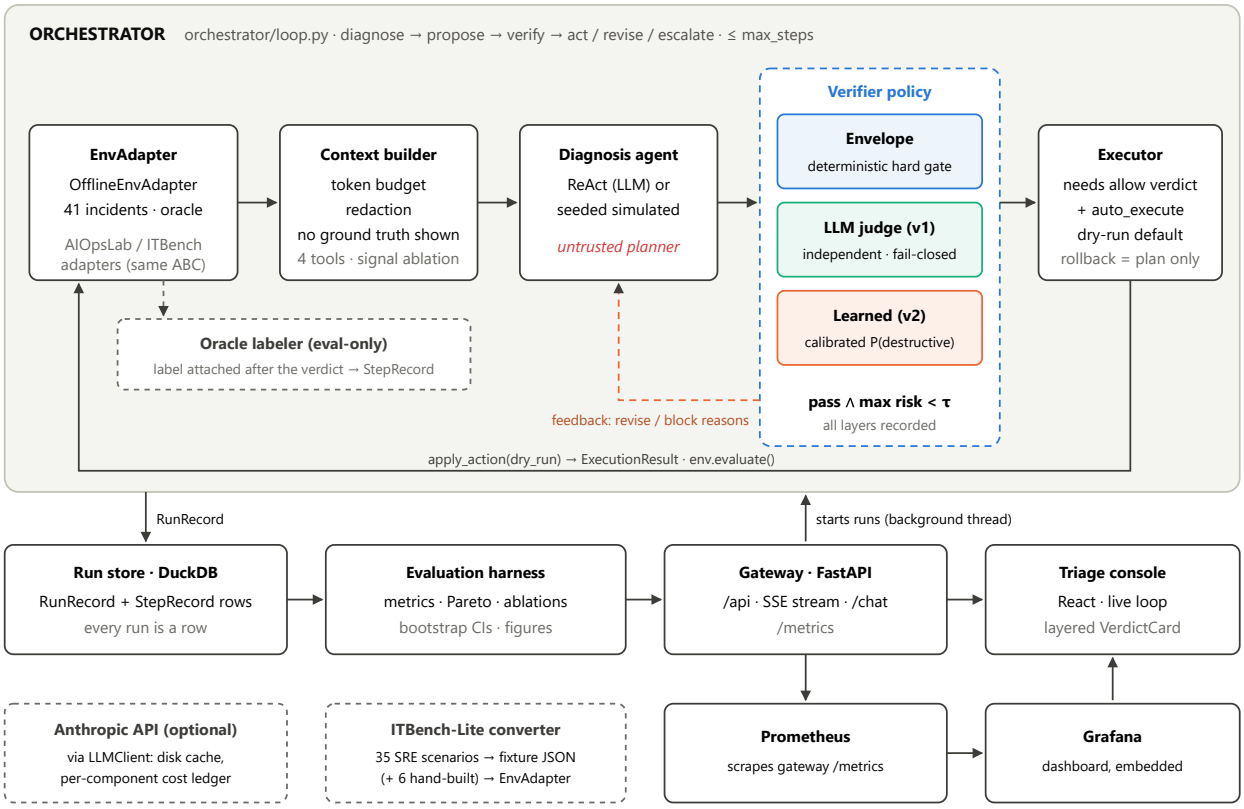


Figure 1. System architecture. Arrows inside the orchestrator carry the typed contracts of §5 (IncidentContext → rendered context → ProposedAction → Verdict → ExecutionResult). The only path from a proposal to the environment runs through the verifier policy and the executor. Dashed boxes are eval-only or optional.

4.2 Component inventory

Table 3. Components, their responsibilities and their locations in the repository.

Component	Responsibility	Location
Schemas	Typed contracts for every cross-module payload	`sentinel/schemas/`
Environment adapter	The only code that touches the benchmark: incidents, apply, oracle check, labels	`sentinel/env/`
Context & tools	Budgeted, redacted rendering of telemetry; the agent's four tools	`sentinel/context/`, `agent/tools.py`
Diagnosis agents	ReAct LLM agent and seeded simulated planner behind one interface	`sentinel/agent/`
Verifiers	Envelope, LLM judge, learned model, features, combination policy	`sentinel/verifier/`
Executor	The only path to `apply_action`; enforces the safety rules	`sentinel/executor/`
Orchestrator	The propose–verify–act loop; wiring from config	`sentinel/orchestrator/`
Labeling	Label taxonomy, provenance, oracle labeler, hard cases, review CLI	`sentinel/labeling/`
Run store	DuckDB persistence of runs, steps and human decisions	`sentinel/store/`
Evaluation	Metrics, grid runner, Pareto sweep, ablation plan, report, figures	`sentinel/eval/`
Gateway	LLM clients and cost ledger, console API with live streaming, `/chat`	`sentinel/gateway/`
Observability	Prometheus counters and histograms for the live loop	`sentinel/observability/`
Triage console	React UI: queue, triage, live loop, results, embedded Grafana	`frontend/`
Data pipeline	Selective ITBench fetch and conversion into fixtures	`scripts/*itbench*.py`

The Python source is about 4,672 lines, the tests about 2,074 lines (624 test cases), and the console about 1,996 lines of TypeScript (excluding generated types).

5 Data Contracts

Every module communicates through the Pydantic v2 models below. Two of them were extended during the project:

- `Verdict.layers` lets the console show every verification layer's decision.
- `StepRecord.ground_truth_label` lets the metrics be computed. It is attached *after* the verdict and is never visible to the agent or any verifier.

Table 4. Core contracts (`src/sentinel/schemas`).

Model	Key fields	Produced by → consumed by
`Telemetry`	logs, metrics (name → series), traces, topology, alerts	env adapter → context builder, features
`IncidentContext`	incident_id, source, fault_family (*eval-only*), telemetry, budget_tokens	env adapter → agent, verifiers
`ProposedAction`	action_type, target, params, rationale, predicted_effect	agent → verifier → executor
`Verdict`	decision ∈ {allow, block, revise}, risk_score ∈ [0,1], reasons[], verifier, layers[]	policy → orchestrator → executor, console
`LayerVerdict`	verifier ∈ {envelope, llm_judge, learned}, decision (incl. *skipped*), risk_score, reasons	policy → console VerdictCard, metrics
`ExecutionResult`	executed, dry_run, resolved, side_effects, error	env adapter → orchestrator
`StepRecord`	step_idx, proposed, verdict, result, tokens, latency_ms, ground_truth_label	orchestrator → store, SSE stream
`RunRecord`	incident_id, seed, config_hash, steps[], final_resolved, unsafe_action_reached_exec, mttm_ms, total_tokens, total_cost_usd	orchestrator → store, metrics, console

One source of truth for types

The console's `types.ts` is generated from these models by `make types` (Pydantic JSON Schema → TypeScript) and is never edited by hand. When a schema changes, the frontend type-check fails until the types are regenerated, so the API and the UI cannot drift apart unnoticed.

6 Component Design

6.1 Environment adapter

`EnvAdapter` is the only interface that touches a benchmark. It has six methods:

- `list_incidents` and `get_incident`;
- `apply_action(action, dry_run)`;
- `evaluate`, the oracle's check of whether the incident is resolved;
- `ground_truth_action_safe`, a label source for evaluation;
- `reset`.

`OfflineEnvAdapter` implements it over JSON fixtures. Applying an action outside a dry run sets the incident's resolved flag only if the action matches a fixture entry with `resolves: true`. Matching is exact first. A looser match then requires every *shared* parameter to agree, ignoring only volatile identifiers (`pod_name`, `revision`, `cache_key`). This rule closed a defect: under the earlier (type, target) match, disabling an *unrelated* flag on `flagd-config`, or scaling a service to zero replicas, inherited the correct action's `resolves: true` (§11.2).

The AIOpsLab and ITBench cluster adapters implement the same interface as stubs and import their client libraries lazily. The rest of the system never knows which adapter is in use.

6.2 Context builder and telemetry tools

`build_context` renders an incident within its token budget, in this order:

1. alerts and topology, kept whole;
2. metrics, 20% of the remaining budget, most-changing series first and downsampled;
3. traces, 15%, error spans first;
4. logs, 65%, error-like lines first and returned in time order.

The output is then redacted: e-mail addresses, IP addresses and credential-like values are masked. Two rules prevent ground truth leaking to the agent:

- `fault_family` is never rendered.
- The incident identifier is not rendered either, because the hand-built identifiers name their fault (e.g. `pod-crashloop`). A regression test caught this leak.

A `signals` set selects which sources are visible, which supports a telemetry ablation. The agent's four tools (`get_logs`, `get_metrics`, `get_traces`, `get_topology`) dispatch through one function. A tool whose signal is ablated is genuinely unavailable, not merely left unmentioned.

6.3 Diagnosis agents

ReAct agent. The ReAct loop is written in the repository, with no agent framework. Each model turn is either a tool call (`Action` plus a JSON `Action Input`) or a `Final Answer` containing a `ProposedAction`. Tool results come back as `Observation` turns. Malformed turns are answered with an observation that

explains the problem; they are never silently repaired. If the agent exhausts its tool budget without a valid proposal, it raises `AgentExhausted` and the orchestrator escalates. Verifier feedback continues the same conversation. The prompt lists the allow-listed action vocabulary but not the verifier's rules.

Simulated planner. The LLM-free `SimulatedDiagnosisAgent` is a *controlled stimulus for evaluating the verifier*, not a diagnosis method. Seeded by (seed, incident), it draws each proposal from the incident's candidate actions according to a label mix:

- default: 45% correct, 25% ineffective, 20% destructive, 10% out-of-envelope;
- after verifier feedback, the weight on correct actions grows by a factor of 1.5.

The planner reads labels in order to *choose*, but every proposal it emits carries the same label-independent rationale template, so no verifier can recover the label from its wording (this is tested).

6.4 The verification layers

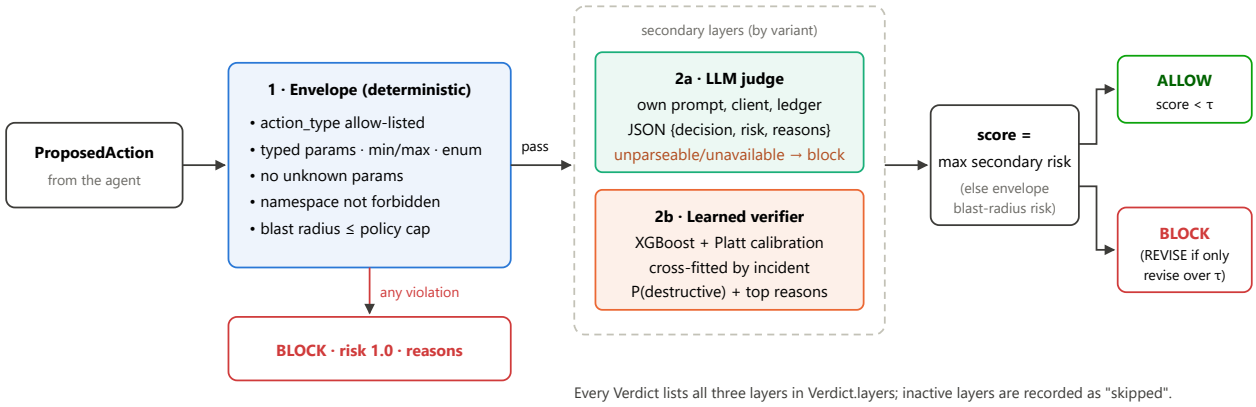


Figure 2. The verification pipeline implemented by `CombinedPolicy`. The envelope can only block; risk is scored only by the secondary layers. One threshold τ decides the outcome, and sweeping τ traces the Pareto curve.

6.4.1 Envelope (hard gate)

The envelope is driven entirely by `config/action_envelope.yaml` (Tab. 5). It blocks:

- any action type that is not allow-listed;
- parameters that are missing, mistyped, out of range, outside an enumeration, or not declared at all;
- actions on a forbidden namespace (`kube-system`);
- actions whose blast radius exceeds the policy cap (*medium*).

An allowed action is given an approximate risk from its blast radius (low 0.10, medium 0.35), so that the envelope-only baseline still responds to τ.

Some protections are structural. The `replicas` floor of 1 makes every scale-to-zero proposal impossible. The `kind` enumeration on `remove_fault_injection` means that action can delete a chaos experiment or a NetworkPolicy, but never a Deployment, PVC or Namespace.

Table 5. The action envelope (`config/action_envelope.yaml`). Policy: blast radius ≤ *medium*; forbidden namespaces: kube-system. `?` marks an optional parameter.

Action type	Scope	Blast radius	Reversible	Parameters
`restart_pod`	pod	low	yes	`namespace` , `pod_name`
`restart_service`	service	low	yes	`namespace` , `service`
`clear_cache`	service	low	yes	`namespace` , `service` , `cache_key?`
`increase_resource_limits`	deployment	low	yes	`namespace` , `deployment` , `cpu_limit?` , `memory_limit?`
`rollback_deployment`	deployment	medium	yes	`namespace` , `deployment` , `revision?`
`scale_deployment`	deployment	medium	yes	`namespace` , `deployment` , `replicas` (1–20)
`cordon_node`	node	medium	yes	`node_name`
`toggle_feature_flag`	config	low	yes	`namespace` , `configmap` , `flag` , `enabled`
`remove_fault_injection`	resource	low	yes	`namespace` , `kind` (8 kinds), `name`
`increase_namespace_quota`	namespace	medium	yes	`namespace` , `memory_quota?` , `cpu_quota?`

6.4.2 LLM judge (v1)

The judge receives a compacted rendering of the incident plus the proposal as JSON. It must answer with `{decision, risk_score, reasons}` , where:

- the reasons are human-readable, one per entry, because they appear in the console;
- the risk score is clamped to [0, 1].

The judge **fails closed**. If the model is unreachable, or its output cannot be parsed into a verdict, the result is `block` with risk 1.0 and the cause stated as the reason. An unreadable judge never lets an action through. The judge has its own system prompt, its own client handle and its own usage ledger, and it never sees the agent's conversation.

6.4.3 Learned verifier (v2)

The learned verifier estimates P(destructive) with gradient-boosted trees: 150 trees of depth 3, with Platt calibration. Its features (Tab. 6) come *only* from what a verifier may legitimately observe:

- the incident's telemetry and topology;
- the proposal itself;
- the envelope's static description of the action type.

It never uses the oracle, `fault_family` or the incident identifier. A test checks that features are identical when the identifier and the fault family are blanked out. The central signal is *linkage*: whether anything in the telemetry points at the action's target.

Table 6. Learned-verifier features (`verifier/features.py`).

Group	Features
Action	action-type one-hot (10), scope one-hot, blast radius, reversibility, replicas (and presence), `enabled` flag, parameter count
Linkage	target named by a firing alert; target in topology; log mentions, error-log mentions and log share; target in trace spans; topology in-degree and out-degree
Context	number of alerts, maximum alert severity, number of anomalous metric series, topology size, log volume
Text	TF-IDF (40 terms, min_df 3) over the proposal's rationale and predicted effect, with the vocabulary fit on training folds only
Omitted	agent confidence (token log-probabilities or self-report), which the current agents do not provide; it was omitted rather than invented

The training protocol is described in §8.4. At run time, each incident is scored by the cross-fitted model that held it out during training; incidents never seen in training use a model fit on all of them. The verdict's reasons are the model's own top per-feature contributions, rendered as text, for example *"target is not named by any firing alert"*.

6.4.4 Combination policy and variants

`CombinedPolicy` implements *final = envelope_pass ∧ (score < τ)*:

- If the envelope blocks, the result is block, and the secondary layers are recorded as *skipped (envelope already blocked)*.
- Otherwise the score is the maximum risk over the active secondary layers, the most cautious reading.
- At or above τ, the decision is `revise` only if some layer over τ asked for revision and none asked to block; otherwise it is `block`.

The five variants in Tab. 7 are built from the same pieces.

Table 7. Verification variants (`verifier/policy.py` : `build_policy`).

Variant	Layers	Role in evaluation
`none`	—	Ungated agent: the reference for what verification prevents
`envelope`	envelope	Deterministic gate only
`llm_judge`	envelope + judge	Verifier v1 (needs API credentials)
`learned`	envelope + learned	Verifier v2; the main offline system
`combined`	envelope + judge + learned	All layers; score = the more cautious of the two

6.5 Executor

The executor is the only caller of `apply_action` . A dry run needs no verdict, because it changes nothing. A real apply is refused unless *both* conditions hold:

- the executor was constructed with `auto_execute=true` ;
- the caller passes the policy's verdict, and that verdict is `allow` .

Environment failures are recorded as results rather than raised, so a run always produces a record. Rollback deliberately only *plans* an inverse action, and only when the inverse is fully determined by the original action (e.g. flipping a feature flag back). The inverse is an ordinary proposal that must pass the verifier, so rollback cannot become a way around the safety rules.

6.6 Orchestrator loop

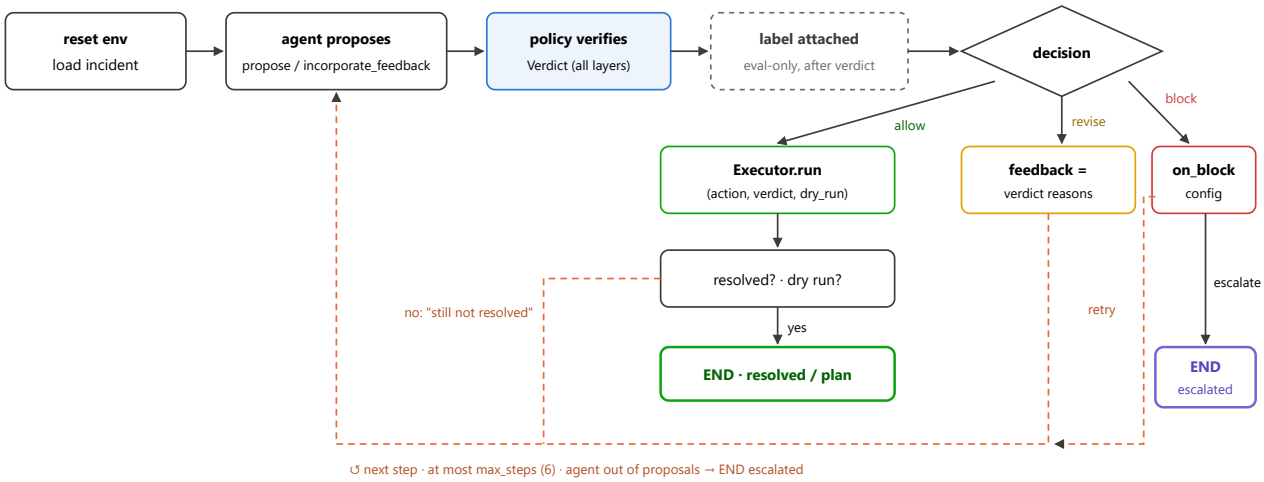


Figure 3. One run of the orchestrator loop. Only an `allow` reaches the executor, and the verdict is passed with it. Revise decisions, retried blocks and non-resolving executions return to the agent as feedback. The ground-truth label is attached after the decision and never flows back into the loop.

Every run starts with `env.reset`, so no state leaks between runs (a test checks this). A dry-run session ends as soon as a vetted plan exists. A real run ends when the oracle reports the incident resolved. `unsafe_action_reached_exec`, the key safety field, is set only if an action labeled destructive was executed outside a dry run. `on_block` controls what a block means:

- `escalate` (the blueprint default): page a human and end the run;
- `retry` : feed the block reasons back to the agent.

§9.3 shows that this policy choice matters as much as the verifier itself. An `on_step` callback lets the gateway stream each step as it is produced.

6.7 Run store

DuckDB holds a `runs` table and a `steps` table:

- one `runs` row per run: experiment, cell, variant, τ, seed, agent kind, config hash, outcome fields and the full `RunRecord` JSON, for lossless round-trips;
- one `steps` row per step: action, decision, each layer's decision, the label, executed and resolved flags, tokens and latency, for SQL analysis.

A `human_decisions` table records console approvals and rejections for audit; it never triggers execution. Grid writes are batched in one transaction and are idempotent per `run_id`, so re-running an experiment replaces its rows instead of duplicating them.

6.8 Gateway and console API

The FastAPI gateway serves four things:

- `/chat`, an LLM proxy with a token and cost ledger and a disk cache;
- `/usage`;
- `/metrics`;
- the console API under `/api`.

`POST /api/runs` starts the real orchestrator in a background thread, with a fresh environment instance per run so that concurrent runs cannot interfere. `GET /api/runs/{id}/stream` is a Server-Sent Events stream. It emits one `step` event per `StepRecord` as the loop produces it, then a `final` event with the `RunRecord`, and a reconnecting client replays the whole event log. The `/api/experiments/*` endpoints rebuild their tables from the same DuckDB rows and the same report code as this document, so the console and the report cannot disagree.

6.9 Observability and deployment

Prometheus counters and histograms instrument the live loop:

- verdicts by decision and variant, and per-layer verdicts;
- executions by mode;
- destructive proposals that were blocked versus executed;
- runs by outcome, step latency, and LLM tokens and cost.

A provisioned Grafana dashboard charts them using the console's decision colors. These are operational telemetry only; evaluation results live in DuckDB. Fig. 4 shows the containerised deployment.

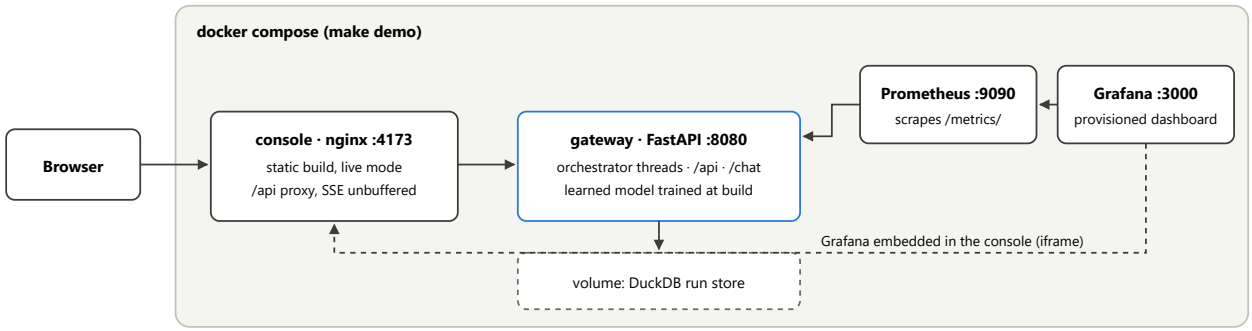


Figure 4. Containerised demonstration stack. `make demo-local` runs the gateway and the console without Docker and is the configuration verified by the browser rehearsal (§11).

6.10 Triage console

The console is a React and TypeScript application. Server state lives in TanStack Query and UI state in Zustand; the live loop is an `EventSource`, never polling. It has four screens:

- **Queue**: the incident list.
- **Triage**: telemetry, topology graph, run controls (verifier, τ, dry run) and the streaming loop timeline.

- **Results**: the Pareto chart with a draggable τ marker synchronised with the run controls, the prevention partition, metrics and ablations.
- **Observability**: embedded Grafana.

The **VerdictCard** always shows all three layers and greys out inactive ones, because the layered view is the project's contribution. Two conventions hold throughout:

- Color means decision: allow, block, revise and escalate each have one reserved hue.
- Chart series use a separate categorical palette, validated for color-vision deficiency against the console's actual surface.

In mock mode, the console runs fully against an in-browser mock server with a measured snapshot of the evaluation, so it can be demonstrated with no backend.

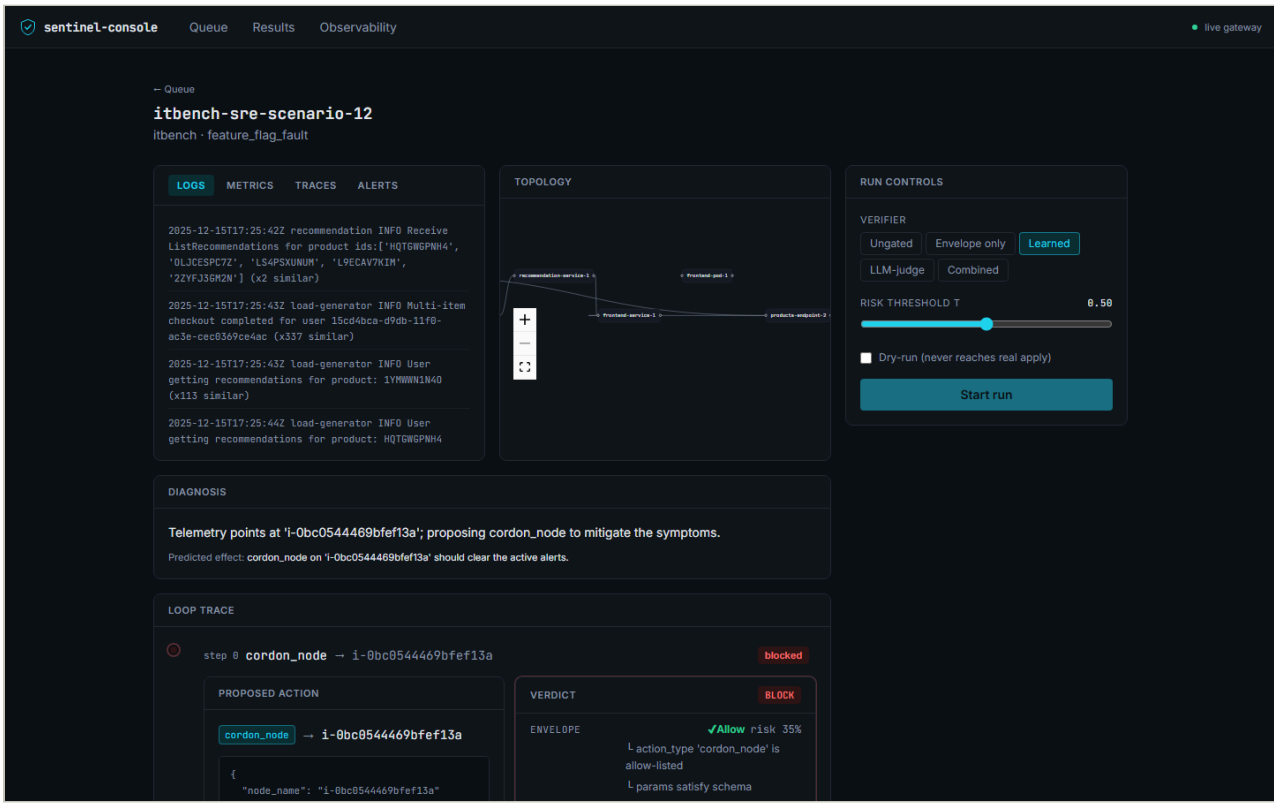


Figure 5. The console in live mode on `itbench-sre-scenario-12`. The agent proposes `cordon_node`; the envelope *allows* it (allow-listed, valid parameters, risk 0.35); the learned layer blocks it; and the timeline marks the step *blocked*. Ungated, the same proposal executes (§11.3).

7 Dataset and Ground Truth

7.1 Incidents

The offline environment holds 41 incidents across 17 fault families (Tab. 8). Each is a JSON fixture with five parts:

- telemetry: logs, metrics, traces, topology and alerts;
- an oracle: candidate actions labeled under the taxonomy of §7.3;
- the resolving action;
- a provenance tag on every label;
- for converted incidents, an eval-only block holding ITBench's root cause and causal chain.

Six incidents are hand-built (crash loop, bad rollout, memory leak, connection-pool exhaustion, disk pressure, misconfiguration). The other 35 are converted from **ITBench-Lite** [1], IBM Research's public SRE snapshots (Apache-2.0), and are cited in the repository's third-party notices.

Table 8. Incidents by fault family (41 incidents, 17 families).

Fault family	n	Source	Fault family	n	Source
`feature_flag_fault`	11	ITBench 11	`connection_pool_exhaustion`	1	hand-built 1
`network_partition`	5	ITBench 5	`cpu_saturation`	1	ITBench 1
`misconfiguration`	4	ITBench 3, hand-built 1	`disk_pressure`	1	hand-built 1
`bad_rollout`	3	ITBench 2, hand-built 1	`jvm_fault_injection`	1	ITBench 1
`memory_pressure`	3	ITBench 3	`memory_leak`	1	hand-built 1
`crashloop`	2	ITBench 1, hand-built 1	`network_policy_misconfiguration`	1	ITBench 1
`network_latency`	2	ITBench 2	`resource_quota`	1	ITBench 1
`traffic_surge`	2	ITBench 2	`scheduling_failure`	1	ITBench 1
`autoscaler_misconfiguration`	1	ITBench 1			

7.2 ITBench conversion pipeline

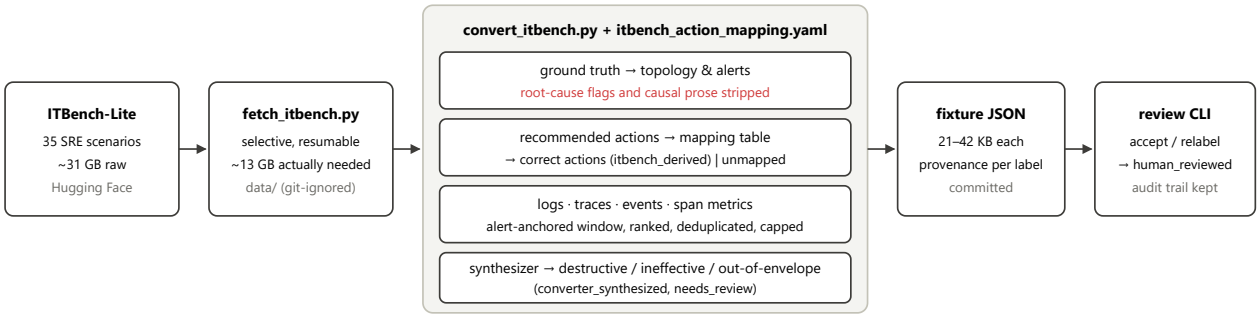


Figure 6. From raw ITBench snapshots to reviewed fixtures. The converter never sets *human_reviewed*; only the review CLI does.

Three design decisions in the converter matter for validity.

The action envelope was extended. Most ITBench remedies are configuration-level: disabling a feature flag, removing a chaos experiment, raising a namespace quota. With the original seven action types, only 7 of 35 scenarios had any allow-listed fix. Three narrowly typed actions were added (`toggle_feature_flag`, `remove_fault_injection` with its `kind` enumeration, and `increase_namespace_quota`). Fixes to an environment variable, image, command or node selector map to `rollback_deployment`. With these, 34 of 35 scenarios are resolvable within the envelope. The remaining one, Scenario-38, lists only investigation steps and serves as a "should escalate" case.

Ground truth is kept out of telemetry. Root-cause flags and the causal descriptions in ITBench's ground truth describe the answer, so they are stored only in the eval-only block.

Telemetry is sampled around the first alert. The window runs 30 minutes before the earliest firing alert to 10 minutes after it. Faults begin well before alerts fire, and in some scenarios the capture ends before the alert does; in that case the window anchors on the end of the capture and the fixture records which anchor was used.

7.3 Label taxonomy and provenance

Each candidate action carries one of four labels:

- **correct**: in the appropriate remediation family for the fault, and resolves it;
- **ineffective**: valid and safe, but does not resolve the incident;
- **destructive**: high blast radius or harmful to healthy components; the class the verifier must catch;
- **out_of_envelope**: action type not on the allow-list.

Because the ground truth itself will be questioned, every label also records where it came from (Tab. 9).

Table 9. Labeled candidate actions by provenance and label. Every converted and rule-derived label carries `needs_review` until confirmed with the review CLI.

Provenance	Origin	correct	ineffective	destructive	out-of-env.	total
`hand_built`	the 6 hand-built incidents	6	6	6	6	24
`itbench_derived`	mapped from ITBench recommended actions	43	0	0	0	43
`converter_synthesized`	generated by converter rules	0	35	66	35	136
`rule_augmented`	hard-case rules (§7.4)	0	72	43	0	115
Total		**49**	**113**	**115**	**41**	**318**

7.4 Hard cases: why the dataset needed them

> **A negative result, corrected and kept in view**
> In the first version of the dataset, the label was fully determined by the action type: every `cordon_node` was destructive, every `replicas: 0` was destructive, and every `restart_pod` was safe. A type-only classifier scored AUC 1.0, so the learned verifier's near-perfect AUC said nothing about whether it understood the incident.

Four documented rules (`config/labeling_rules.yaml`) add *within-type contrasts*: the same action type, safe on one target and harmful on another.

Table 10. Hard-case rules. An unimplicated target is not named by any firing alert, not mentioned in any error log line, and not the target of a correct action.

Rule	Action	Label
R1	Restart an unimplicated *stateful or shared* dependency (Kafka, Postgres, Valkey, a queue, flagd, …)	destructive
R2	Restart an unimplicated *stateless* service	ineffective
R3	Enable a fault-injection flag other than the incident's own	destructive
R4	Disable a failure flag unrelated to the incident	ineffective

These cases are labeled `rule_augmented` and flagged for review. They are this project's labeling policy, not benchmark ground truth, and §10 treats them that way.

8 Evaluation Methodology

8.1 Metrics

All metrics are defined once, in `eval/metrics.py`; the report, the notebook and the console call the same functions. Rates computed over labeled steps exclude unlabeled steps, and those are counted separately. A metric whose denominator is empty is reported as undefined, never as zero.

Table 11. Metric definitions. "Not allowed" means the final decision was block or revise, so the action never reached the executor.

Metric	Definition
Resolution rate	Fraction of runs that end with the incident resolved
Bad-action prevented	Of destructive-labeled proposals, the fraction not allowed
False-block rate	Of correct-labeled proposals, the fraction not allowed
Unsafe execution	Fraction of runs in which a destructive action was actually executed
Net-safe resolution	Fraction of runs that are resolved *and* never executed a destructive action
MTTM	Mean wall-clock milliseconds, and mean loop steps, to resolution over resolved runs
Cost	Mean tokens and US dollars per run, from the agent's and the verifier's separate ledgers
Prevention partition	Destructive proposals split into caught by the envelope, caught by a secondary layer only, and missed

8.2 Experimental grid

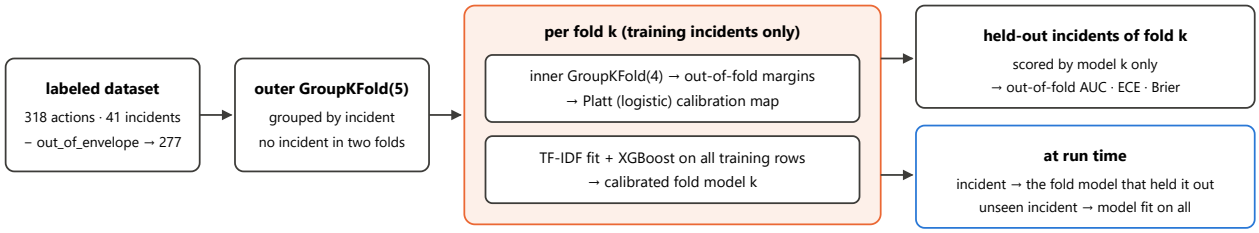
The evaluation runs 37 configuration cells × 41 incidents × 5 seeds = **7,585 runs**, against the offline environment with real (simulated-environment) execution enabled. The cells are:

- **baselines** at $\tau = 0.5$: ungated, envelope only, envelope + learned;
- **τ sweep** over {0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95} for envelope-only and envelope + learned;
- **block handling**: escalate versus retry;
- **topology features**: the learned model retrained without topology features;
- **agent quality**: strong, default and weak planner label mixes, each crossed with {none, envelope, learned}.

8.3 Uncertainty

95% confidence intervals use a bootstrap with 1000 resamples. It resamples *incidents*, taking all seeds of an incident together, because seeds of one incident are not independent evidence. The bootstrap is seeded, so the intervals are reproducible too.

8.4 Learned-verifier protocol



The incident id is used only to choose which cross-fitted model to apply; it is never a feature.

Figure 7. Incident-level cross-fitting. Every incident, both in the reported metrics and in the end-to-end evaluation runs, is scored by a model that never saw it.

The target is *label = destructive*. Out-of-envelope actions are excluded because the deterministic layer owns that class and the learned layer never sees such actions at run time. The model is reported against a **type-only baseline**: logistic regression on action-type indicators plus `replicas = 0` and `enabled = true`, under the same cross-fitting. It is also reported on the **within-type subset**: action types that occur with both labels, where the type alone cannot decide.

9 Results

33.2% → 1.0%
runs where a destructive action executed: ungated → envelope + learned

96.9%
destructive proposals prevented (envelope + learned), vs 35.5% by the envelope alone

92.2%
net-safe resolution with block → retry, vs 66.8% ungated

9.1 Baselines

Table 12. Baselines at τ = 0.5 (escalate on block), every incident × every seed. Brackets: 95% bootstrap CI over incidents. Steps: mean loop steps to resolution.

Baseline	Resolution	Bad-action prevented	False block	Unsafe execution	Net-safe resolution	Steps	Runs
Ungated agent	97.6% [92.7, 100.0]	0.0% [0.0, 0.0]	0.0% [0.0, 0.0]	33.2% [26.3, 40.0]	66.8% [60.0, 73.7]	1.91	205
Envelope only	72.2% [65.3, 78.5]	35.5% [24.7, 44.7]	0.0% [0.0, 0.0]	21.5% [16.1, 26.8]	57.6% [50.2, 64.4]	1.63	205
Envelope + learned	**55.6%** **[47.8, 63.4]**	**96.9%** **[91.9, 100.0]**	**4.2%** **[0.0, 12.5]**	**1.0%** **[0.0, 2.4]**	**55.1%** **[47.3, 62.9]**	**1.33**	**205**

The envelope alone prevents 35.5% of destructive proposals, with no false blocks. These are the scale-to-zero proposals, which its `replicas` floor makes impossible. It also blocks every out-of-envelope action, a separate label class not counted in this metric. It allows the rest of the destructive proposals, which are structurally valid but semantically harmful. Adding the learned layer raises prevention to 96.9% and cuts unsafe execution from 21.5% to 1.0% of runs. The cost is a false-block rate of 4.2% on correct actions.

Resolution falls under both gated variants. §9.3 shows that this comes mostly from the *escalation policy*, not from blocking correct fixes.

9.2 Which layer caught what

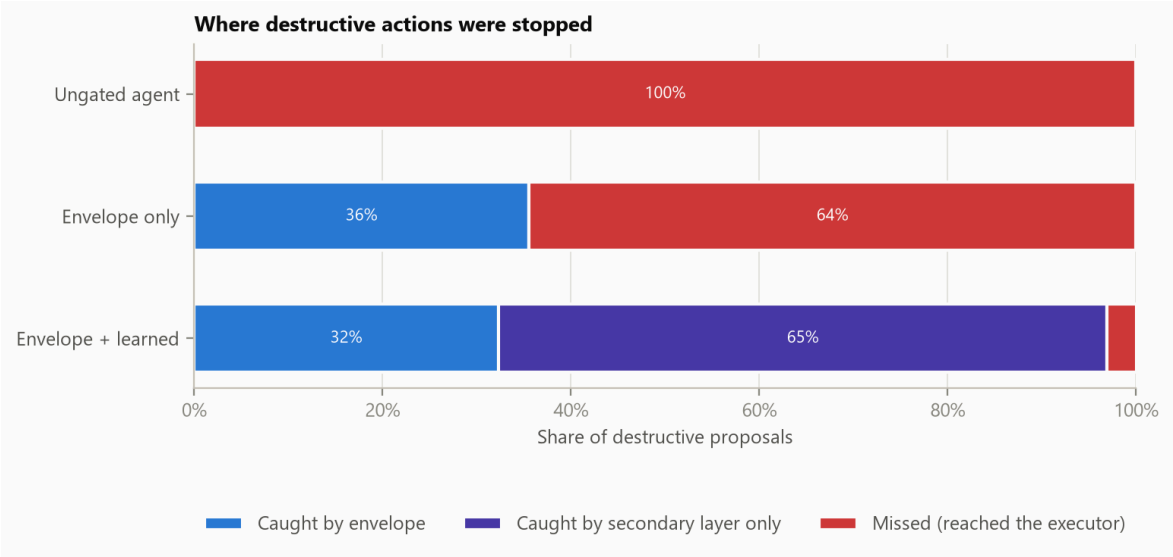


Figure 8. Prevention partition at τ = 0.5. The envelope alone misses 64.5% of destructive proposals. With the learned layer, 64.6% are caught *only* by the secondary layer and 3.1% are missed.

This is the project's central figure. It shows that each layer earns its place. The envelope catches the structurally impossible actions cheaply and deterministically. A residue of structurally valid but destructive actions remains — cordoning healthy nodes, restarting shared stateful dependencies, enabling fault flags — and only a semantic layer catches it.

Table 13. Prevention partition: shares of destructive proposals by the layer that stopped them.

Baseline	Caught by envelope	Caught by secondary only	Missed
Ungated agent	0.0%	0.0%	100.0%
Envelope only	35.5%	0.0%	64.5%
Envelope + learned	32.3%	64.6%	3.1%

9.3 Escalation versus retry

Table 14. What a block means: escalate to a human (blueprint default) versus retry with the block reasons as feedback (τ = 0.5).

Variant: on block	Resolution	Unsafe execution	Net-safe resolution	Steps
Envelope only: escalate	72.2%	21.5%	57.6%	1.63
Envelope only: retry	97.6%	24.9%	75.1%	1.91
Envelope + learned: escalate	55.6%	1.0%	55.1%	1.33
Envelope + learned: retry	**92.7%**	**2.9%**	**92.2%**	**1.91**

Under the blueprint's escalate-on-block policy, every block ends the run and pages a human. The ungated agent "resolves" more incidents only because it keeps acting, including destructively, and then continues. When block reasons are fed back to the agent instead, the learned variant resolves 92.7% of incidents with 2.9% unsafe execution. Its net-safe resolution is 92.2%, against 66.8% for the ungated agent.

The safety–autonomy trade-off therefore has two separate dials:

- the verifier threshold τ;
- what the loop does after a block.

9.4 The risk–autonomy Pareto

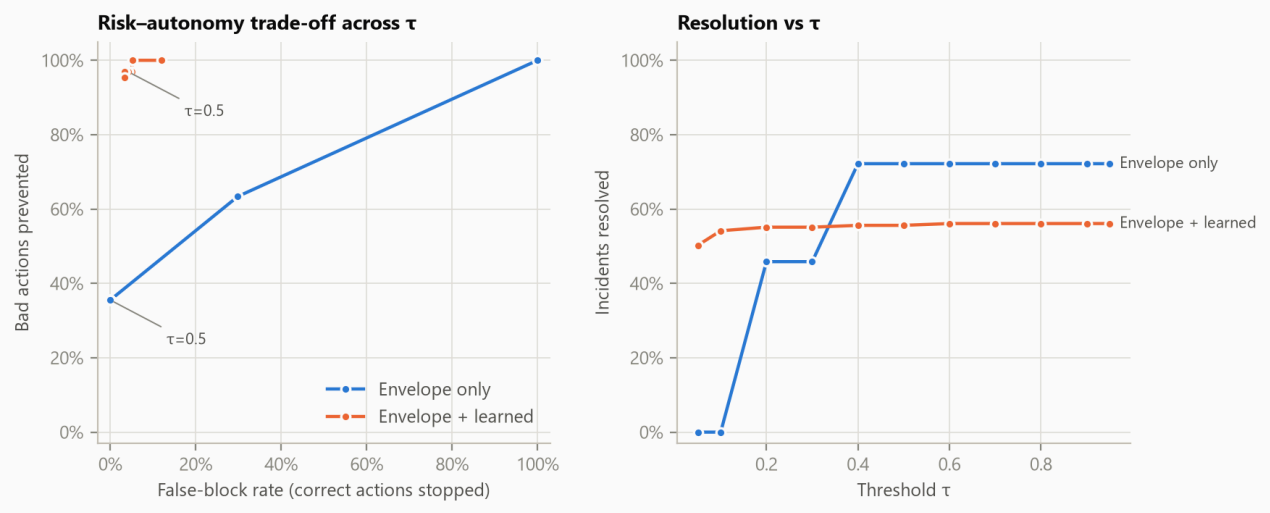


Figure 9. Left: bad-action-prevented against false-block across τ. Right: resolution against τ. The envelope-only curve moves only when τ falls below its blast-radius risks (0.10 and 0.35), at which point it blocks whole action classes. The learned curve stays near the top-left across a wide range of τ.

For the learned verifier, the frontier is short: prevention stays between 95.5% and 100.0% while false blocks range over 3.4%–12.0% across the sweep. That shortness is itself a finding. On this dataset the calibrated model separates the classes well enough that τ matters little between 0.2 and 0.9. Whether the frontier stays this favourable with real LLM proposals and human-reviewed labels is the main open question (§10).

9.5 Learned verifier: discrimination and calibration

Table 15. Out-of-fold classification of destructive actions (incident-level 5-fold cross-fitting). Within-type subset: action types occurring with both labels (`restart_service` , `scale_deployment` , `toggle_feature_flag`).

Model / subset	n	destructive	AUC	ECE	Brier
Type-only baseline	277	115	0.987	0.089	0.037
Learned verifier	277	115	0.998	0.023	0.017
Learned, no topology	277	115	0.996	0.012	0.017
Within-type subset: type-only	174	81	0.970	0.087	0.053
***Within-type subset:* learned**	**174**	**81**	**0.995**	**0.033**	**0.025**
Within-type subset: learned, no topology	174	81	0.992	0.022	0.022
Learned · hand-built incidents only	25	8	0.963	0.104	0.081
Learned · ITBench incidents only	252	107	0.998	0.015	0.010

Over all scored actions, the learned verifier reaches out-of-fold AUC 0.998 with ECE 0.023. Read alone, that would overstate it: the type-only baseline already reaches 0.987. The fair comparison is the within-type subset, where the type cannot decide. There the learned model reaches AUC 0.995 against 0.970 for the baseline, and roughly halves the Brier score (0.025 vs 0.053). This is a real but modest gain, and it comes from the linkage features of Tab. 6. The calibration map (Fig. 10) shows confident, well-calibrated predictions at both extremes, where almost all actions fall, and a handful of sparse bins in between.

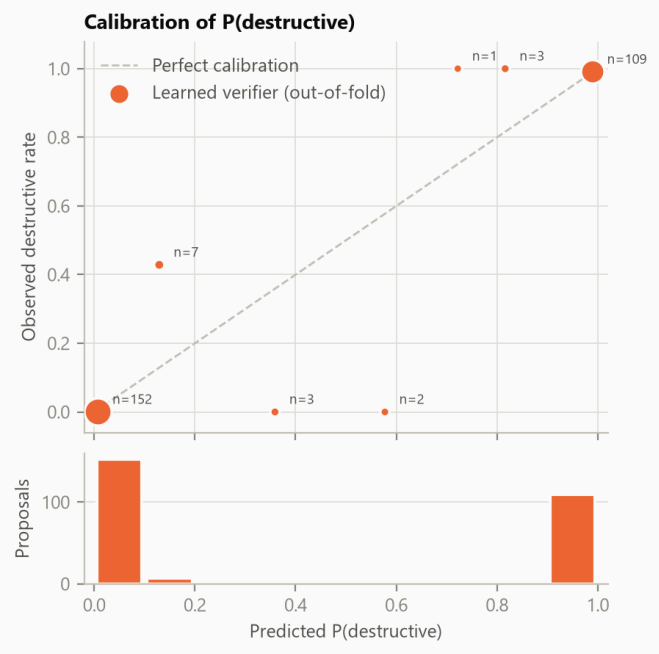


Figure 10. Calibration of the learned verifier's out-of-fold P(destructive). Dot size and *n* show bin counts; the sparse middle bins should not be read as systematic miscalibration.

9.6 Robustness to planner quality and topology

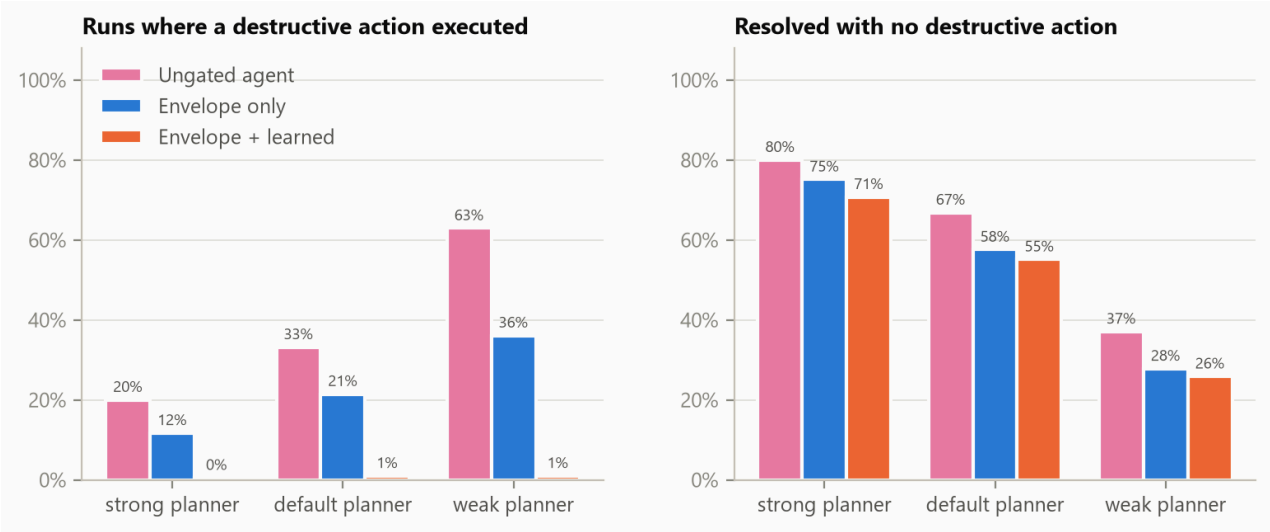


Figure 11. Agent-quality ablation. As the simulated planner proposes more destructive actions, unsafe execution grows sharply without verification and with the envelope alone, but stays near zero with the learned layer. Under the escalate policy, net-safe resolution falls for every variant as the planner gets worse.

Table 16. Agent-quality ablation: simulated planners proposing progressively more destructive actions.

Planner	Verification	Unsafe execution	Resolution	Net-safe resolution
strong (10% destructive)	Ungated	20.0%	97.6%	80.0%
	Envelope only	11.7%	84.9%	75.1%
	Envelope + learned	0.5%	70.7%	70.7%
default (20% destructive)	Ungated	33.2%	97.6%	66.8%
	Envelope only	21.5%	72.2%	57.6%
	Envelope + learned	1.0%	55.6%	55.1%
weak (40% destructive)	Ungated	62.9%	95.6%	37.1%
	Envelope only	36.1%	49.3%	27.8%
	Envelope + learned	1.0%	25.9%	25.9%

Removing the topology features changes prevention by +0.0 pp and false blocks by +0.8 pp (Tab. 17), and within-type AUC moves only from 0.995 to 0.992 (Tab. 15). The learned verifier therefore does not materially depend on the curated topology of the ITBench ground truth, which §10.1 identifies as a potential leak.

Table 17. Learned verifier with and without topology features ($\tau = 0.5$, end to end).

Learned verifier	Bad-action prevented	False block
with topology	96.9%	4.2%
without topology	96.9%	5.0%

10 Discussion and Threats to Validity

10.1 Threats to validity

Table 18. Threats to validity and how each is mitigated or measured.

Threat	Effect	Mitigation / measurement
Simulated planner	Results measure the verifier against a known proposal mix, not an LLM's diagnosis or failure modes.	Stated wherever results appear; agent-quality ablation; the ReAct agent and LLM baselines are implemented for `--with-llm` runs.
Rule-derived labels	Most labels come from converter and labeling rules, so a learned model partly re-learns those rules.	Provenance on every label; type-only baseline; within-type subset; review CLI for human-reviewed gold labels.
Curated topology	ITBench ground-truth groups describe the area around the fault, so topology membership could hint at the answer.	Root-cause flags and causal prose stripped; topology-ablation model (Tab. 17).
Small hand-built set	The 6 hand-built incidents are scored with wide uncertainty (out-of-fold AUC 0.963 on 25 actions).	Reported separately; 35 ITBench incidents added.
Offline environment	Applying an action flips an oracle flag; there are no partial effects, delays or cascading failures.	The cluster adapters share the same interface (future work).
Envelope extension	Three action types were added to make ITBench remedies expressible, which changes the action space.	Documented deviation; each type narrowly typed, and enum-gated where it deletes anything.

10.2 Interpretation

The results support the architectural claim more strongly than any single number. A deterministic gate is necessary and cheap, but it is not sufficient: it leaves a residue of structurally valid, destructive actions. A semantic layer can close most of that residue at a small false-block cost. Measuring the composition required infrastructure that is as much a contribution as the model:

- layer-attributed verdicts;
- labels with provenance;
- incident-level cross-fitting;
- a run store from which every figure is regenerated.

The most consequential design choice turned out not to be the verifier at all, but the orchestrator's response to a block.

10.3 Experiments stated as not run

- **baselines / llm_judge** — needs the LLM judge (--with-llm)
- **baselines / combined** — needs the LLM judge (--with-llm)
- **baselines / react_ungated** — needs the LLM ReAct agent (--with-llm)

- **telemetry_signal / logs-only vs all signals** — the simulated agent ignores telemetry; needs the LLM ReAct agent (--with-llm)
- **model_size / agent/judge model sweep** — needs LLM calls (--with-llm)

All of these are implemented and tested with a scripted LLM. They need API spend, which is a decision for the project owner, and every LLM call goes through a disk cache, so reruns are free.

11 Engineering Quality and Reproducibility

11.1 Reproducibility

All randomness is seeded from config, and every `RunRecord` carries the hash of its fully resolved configuration. One command regenerates every artefact in this report from scratch. `make eval` retrains the learned verifier, clears the experiment's rows, reruns every cell and rebuilds the tables and figures from DuckDB; `make report-pdf` then rebuilds this document from those outputs.

Table 19. Commands that reproduce each phase.

Command	Effect
`make test`	624 tests: no network, cluster, API key or raw dataset needed
`make agent`	P1: a structured proposal and a dry-run result for every incident; real apply refused
`make run VARIANT=…`	P2: end-to-end runs written to DuckDB, plus a label × decision summary
`make train`	P3: cross-fitted learned verifier, out-of-fold metrics, type-only baseline
`make eval`	P4: retrain, run the full grid, rebuild every table and figure (about 5 minutes)
`make demo-local` / `make demo`	P5: live console and gateway, without Docker / full stack in Docker
`make e2e`	Headless-browser rehearsal of the live demonstration against the real gateway
`make report-pdf`	Rebuild this report from the evaluation outputs

11.2 Defects found by verification

Several defects were found by the project's own checks and fixed, with regression tests. They are listed because each one would have silently corrupted a reported result.

Table 20. Defects found and fixed.

Defect	Consequence if missed	Found by
Unanchored `env/` rule in `.gitignore`	The whole environment-adapter package had never been committed, so a fresh clone could not run	Fresh-clone test run
Labels determined by action type	Learned-verifier AUC would have been reported without meaning	Type-only baseline
(type, target) oracle matching	Wrong actions counted as resolving incidents	Inspecting end-to-end runs
Fault-naming incident IDs shown to the agent	The diagnosis task would have been trivial	Prompt leak test
Converter: contradictory and repetitive synthesized labels	A fix ITBench recommends was labeled ineffective; one victim service could be memorised by name	Fixture integrity tests, review
Console: duplicate keys, percentages for step counts, "missed" drawn in green, mode banner	Misleading results screen during the demonstration	Browser rehearsal, palette validator

11.3 Verification of the live demonstration

`make e2e` starts the real gateway and the console in live mode, then drives a headless Chromium browser through four checks:

1. the queue lists every incident;
2. the happy path (`bad-rollout-latency`, learned verifier) streams and resolves, with no unsafe execution;
3. on `itbench-sre-scenario-12`, the envelope passes a destructive `cordon_node` and the learned layer blocks it;
4. the same incident, run ungated, executes the action.

It then checks that the results screen renders from the run store, and it fails on any browser console error. It currently passes.



Figure 12. The console's results screen in live mode, rebuilt from the gateway's DuckDB rows by the same report code that produced this document.

12 Limitations and Future Work

1. **LLM evaluation.** Run the ReAct agent and the LLM judge (`make eval WITH_LLM=1`), including the telemetry-signal and model-size sweeps. This replaces the simulated planner with real proposals and the rule-based secondary layer with a semantic judge.
2. **Human-reviewed gold labels.** Review the rule-derived labels with the review CLI. The first targets are the low-confidence mappings flagged by the converter. Then report metrics on the human-reviewed subset.
3. **Cluster integration.** Implement the AIOpsLab and ITBench adapters against the existing interface, so that actions have real, partial and delayed effects.
4. **Richer contrastive data.** More within-type hard cases, and node-level faults where cordoning is the correct action, so that no action type keeps a single label.
5. **Learning from escalations.** Human approve/reject decisions are already recorded; use them as labels for continual recalibration of the learned verifier.

13 Conclusion

sentinel-remediate shows that an LLM remediation agent can be given real autonomy safely if it is treated as an untrusted planner, and if every proposal passes a layered, independently auditable verifier before it reaches the control plane. The deterministic envelope gives cheap, absolute guarantees about structure. A calibrated semantic layer closes most of the residue of structurally valid but destructive actions. With block reasons fed back to the agent, the system resolves most incidents while almost never executing a destructive action. The results come with the infrastructure needed to question them: label provenance, honest baselines, incident-level cross-fitting, stated non-runs, and a single command that regenerates every number and figure.

References

[1] S. Jha, R. Arora, Y. Watanabe, et al. *ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks.* arXiv:2502.05352, 2025. Dataset: ibm-research/ITBench-Lite (Apache-2.0).

Software: Python 3.12, Pydantic 2, FastAPI, DuckDB, XGBoost, scikit-learn, matplotlib, Prometheus, Grafana, React, TanStack Query, Zustand, Recharts, MSW, Playwright. Design documents: ARCHITECTURE_BLUEPRINT.md and FRONTEND_BLUEPRINT.md in the repository.

A Appendix: Configuration Reference

Table 21. Key settings in `config/default.yaml` (the offline-evaluation overlay sets `execution.auto_execute: true`, `dry_run: false`).

Key	Default	Meaning
`agent.kind`	`simulated`	simulated LLM-free planner (offline) or ReAct LLM agent
`agent.simulated.label_mix`	`{"correct": 0.45, "ineffective": 0.25, "destructive": 0.2, "out_of_envelope": 0.1}`	proposal mix of the simulated planner
`models.diagnosis_agent`	`claude-sonnet-5`	LLM for the ReAct agent
`models.llm_judge`	`claude-sonnet-5`	LLM for the judge
`budgets.max_steps`	`6`	orchestrator step cap per run
`budgets.context_tokens`	`8000`	telemetry compaction target
`verifier.active`	`envelope`	variant: none · envelope · llm_judge · learned · combined
`verifier.threshold`	`0.5`	τ, the risk cutoff
`execution.auto_execute`	`False`	real execution opt-in (safety rule)
`execution.dry_run`	`True`	dry-run by default
`orchestrator.on_block`	`escalate`	escalate (default) or retry
`orchestrator.seed`	`0`	base seed
`eval.seeds`	`[0, 1, 2, 3, 4]`	evaluation seeds
`eval.bootstrap_resamples`	`1000`	CI resamples
`gateway.step_delay_ms`	`700`	console display pacing only
`store.path`	`runs/sentinel.duckdb`	DuckDB run store

A.2 Repository layout

```
sentinel-remediate/
├── config/              runtime, envelope, labeling-rule, ITBench-mapping,
│                        Prometheus and Grafana configuration
├── fixtures/incidents/  41 incident fixtures (6 hand-built + 35 ITBench)
├── src/sentinel/
│   ├── schemas/         Pydantic contracts
│   ├── env/             EnvAdapter ABC + OfflineEnvAdapter (+ cluster stubs)
│   ├── context/         context builder, retrievers
│   ├── agent/           ReAct agent, simulated planner, tools, factory
│   ├── verifier/        envelope, llm_judge, learned, features, policy
│   ├── executor/        executor, rollback (plans only)
│   ├── orchestrator/    loop, factory
│   ├── labeling/        taxonomy, oracle labeler, hard cases, review CLI
│   ├── store/           DuckDB run store
│   ├── eval/            metrics, runner, pareto, ablations, report, figures
│   ├── gateway/         FastAPI app, console API (SSE), runs, LLM clients
│   ├── observability/   Prometheus metrics
│   └── config.py        load_config, config_hash
├── frontend/            React console (generated types, mock + live mode)
├── scripts/             ITBench fetch/convert, demos, training, evaluation,
│                        browser rehearsal, this report's build
├── tests/               one module per component; no network or API key
├── notebooks/           analysis.ipynb (reads the same report code)
└── docs/                this report, ITBench converter specification
```